

Typed Symbolic Regression for Quaternion Integration Discovery

1. Introduction

Symbolic regression (SR) is the task of discovering a mathematical expression that fits observed data. Classical SR (e.g., via genetic programming) operates over untyped expression trees, which means the search space includes many syntactically valid but semantically meaningless combinations — adding a scalar to a vector, applying a rotation to a number. These type errors slow convergence and inflate tree complexity.

We present a typed symbolic regression system built in Haskell using a Generalised Algebraic Data Type (GADT) expression tree. Every node in the tree is statically indexed by its input type and output type. Mutations always produce well-typed trees, and the function pool is partitioned into type-specific bins, so the search never wastes effort on ill-typed candidates.

The target problem is discovering the quaternion integration formula from motion capture data:

$$\Delta q = \exp_q(0.5 \cdot dt \cdot \omega)$$

where $\omega \in \mathbb{R}^3$ is angular velocity and $dt \in \mathbb{R}$ is the variable timestep. The output Δq is a unit quaternion. This problem involves three distinct types (scalar, vector, quaternion) and cannot be expressed cleanly in a single-type SR system without ad-hoc encoding.

The main contributions are: (1) a multi-typed GADT expression tree with constant nodes for each type, (2) parallel tempering MCMC search with wall-time stopping, (3) a constant folding pass to remove dead branches, and (4) an empirical evaluation showing the system reliably discovers the exact formula from 3000 synthetic data points.

2. Related Work

Standard genetic programming (Koza 1992) uses untyped tree evolution; strongly-typed GP (Montana 1995) introduced type constraints but was rarely used with rich algebraic types.

DataHaskell provides a Haskell-native SR library using polynomial regression and expression enumeration. It operates over a single numeric type, making it ill-suited for vector/quaternion domains.

Parallel tempering (replica exchange) is a well-known technique from statistical physics, applied here to MCMC over tree space. Multiple chains run at different temperatures; high-temperature chains explore broadly while low-temperature chains refine promising solutions.

E-graphs (hegg library for Haskell) offer efficient equality saturation for term rewriting. However, the multi-typed GADT is single-sorted only via flattening, making e-graph integration non-trivial; constant folding via tree evaluation was used instead.

3. Method

3.1 PTree GADT

```
data PTree a b where
  PBin    :: (Typeable x, Typeable y) => String -> (x -> y -> b) -> PTree a x -> PTree a y ->
  PUn     :: Typeable x => String -> (x -> b) -> PTree a x -> PTree a b
  PV      :: String -> (a -> b) -> PTree a b
  PConst  :: IORef Double -> PTree a Double
  PConstV :: Vec3 -> PTree a Vec3
  PConstQ :: Quat -> PTree a Quat
```

PTree a b represents a computation from input type a to output type b. Leaf nodes PV project from the input record. Constant nodes (PConst, PConstV, PConstQ) hold mutable references optimised during search.

Type matching uses Data.Typeable and eqT to check at runtime whether a candidate function's argument type matches the required type. This allows the function pool to be stored in a heterogeneous list while remaining type-safe.

3.2 Function Pool

For the quaternion integration problem, the typed function pool is:

Kind	Name	Type
Binary	(*)	Double→Double→Double
Binary	scale_v	Double→Vec3→Vec3
Binary	add_v	Vec3→Vec3→Vec3
Binary	q_mul	Quat→Quat→Quat
Unary	exp_q	Vec3→Quat
Unary	neg_v	Vec3→Vec3
Unary	neg	Double→Double
Leaf	omega	MocapInput→Vec3
Leaf	dt	MocapInput→Double

The norm_v :: Vec3 → Double function was deliberately excluded from the pool after analysis revealed it enables a degenerate solution exploiting approximately-constant $\|\omega\|$ (failure mode F2).

3.3 Parallel Tempering Search

The search uses 12 chains at log-spaced temperatures from 0.001 to 10.0. Each “round” consists of 200 independent Metropolis steps per chain (run in parallel via async), followed by adjacent-chain swap proposals with Boltzmann acceptance. Chains use independent Mersenne Twister generators.

SearchConfig defaults:

```
chains: 12, temps: logspace(0.001, 10.0)
max rounds: 5000, steps/swap: 200
target energy: 5e-5
tree depth: 3
```

3.4 Energy Function

Loss is the mean dot-product quaternion distance over a 300-point subsample:

$$L = (1/N) \sum (1 - |\hat{q}_i \cdot q_i|^2)$$

This handles the quaternion double-cover (q and $-q$ represent the same rotation) and is smooth everywhere.

3.5 Constant Folding

After search, foldConstants recursively checks for subtrees containing no PV leaves and evaluates them to a single constant node. This collapses dead branches (e.g., `x add_v neg_v(x)`) before displaying the result.

4. The Mocap Experiment

4.1 Problem Setup

Given a sequence of angular velocities ω and variable timesteps dt , predict the incremental quaternion rotation Δq . The ground truth is:

$$\Delta q = \exp_q(\omega \cdot dt / 2) \quad \text{where} \quad \exp_q(v) = \cos(\|v\|) + \sin(\|v\|)/\|v\| \cdot \hat{v}$$

4.2 Data Generation

Synthetic data is generated by `scripts/generate_synthetic_bvh.py`. Key design choices to prevent degenerate solutions:

1. **Variable $\|\omega\|$:** An amplitude envelope $1 + 7 \cdot (0.5 + 0.5 \cdot \sin(2\pi \cdot 0.05 \cdot t))$ makes $\|\omega\|$ vary from ~ 0.1 to ~ 15 rad/s. This prevents exploiting constant $\|\omega\|$ (failure mode F2).
2. **Variable dt :** Timesteps are drawn uniformly from $[1/200, 1/50]$ s (50-200 Hz). This prevents absorbing a constant dt into scalar coefficients (failure mode F3).
3. **3000 frames:** Provides sufficient statistical coverage. Search uses a 300-point subsample; final evaluation uses all 2999 rows.

Round-trip validation confirms mean error $< 1e-6$ (floating-point precision).

5. Results

5.1 Iteration Table

Iter	Change	Runs	Success	Success Rate	Dominant Failure	Note
1	baseline (depth=5)	3	2/3	67%	F6 (ApproxExtraDt)	2 clean, 1 has extra (1-dt) factor
2	depth=3	3	2/3	67%	F6 (ApproxExtraDt)	Depth reduc- tion insuffi- cient
3	remove add_v	3+10	12/13	92%	F7 (QMulComplex)	Rare q_mul overfit
4	remove q_mul	10	10/10	100%	none	Perfect con- ver- gence
5	+simplifyTrees pass	10	10/10	100%	none	Cleaner output trees

Representative found trees (normalised): `- exp_q((0.5 * dt) scale_v omega)` — cleanest form (iterations 4-5) `- exp_q(dt scale_v (0.5 scale_v omega))` — scalar order variant, equivalent `- exp_q(0.5 scale_v neg_v(dt scale_v neg_v(omega)))` — double-neg wrapper

5.2 Failure Modes Observed

ID	Name	Example	Cause	Fix Applied
F6	ApproxExtraDt	$\exp_q(c \cdot dt \cdot (1 - dt) \cdot \omega)$	add_v allows $\omega + dt \cdot (-\omega) = (1 - dt) \cdot \omega$; extra dt term vanishes for small dt	Removed add_v
F7	QMulComplex	$\exp_q(\omega) * A * \exp_q(-\omega) * \exp_q(0.5 \cdot dt \cdot \omega)$	q_mul enables complex quaternion chains that fit data via conjugation identity	Removed q_mul

Failure modes F1-F5 (from prior work) were prevented by data design choices before iteration 1: variable $\|\omega\|$ (no F2), variable dt (no F3), no norm_v in pool (no F2), sufficient rotation magnitude (no F1).

6. Discussion

6.1 What the Typed Approach Prevented

In a standard untyped GP system operating on a flat vector of numbers, representing the mixed-type formula $\exp_q(0.5 \cdot dt \cdot \omega)$ requires encoding: quaternions as 4-vectors, angular velocities as 3-vectors, and scalars as single numbers. All arithmetic must be typed by convention and enforced post-hoc. In practice, this means the search wastes significant effort on expressions like $(\omega + dt)$ (adding a vector to a scalar) or $\exp_q(q_prev)$ (applying the quaternion exponential to an already-quaternion input). Our GADT representation makes these expressions literally unrepresentable — they would be a Haskell type error.

This matters empirically: the search space for a depth-3 Quat-output tree over 7 function symbols is much smaller when ill-typed trees are excluded. The correct formula is $\exp_q(\text{scale}_v(\text{scale}(*)(c, dt), \omega))$ — a specific 4-node tree — and it was found reliably within hundreds of rounds.

6.2 What Function Pool Design Revealed

The most instructive finding is that the search reliably exploited degenerate solutions when given functions that made such exploitation easy:

1. **norm_v (removed pre-experiment):** Allows $\|\omega\| \approx \text{const}$ exploitation (F2). If $\|\omega\|$ is approximately constant across the dataset, the tree can encode it as a constant and find a simpler-looking but wrong formula.
2. **add_v (removed in iteration 3):** Allows $\omega + c \cdot dt \cdot (-\omega) = (1 - c \cdot dt) \cdot \omega$ exploitation (F6). The extra dt factor is invisible for small dt, so the loss is low but the formula is wrong.
3. **q_mul (removed in iteration 4):** Allows conjugation-based equivalent formulas: $\exp_q(v) * Q * \exp_q(-v)$ computes a rotation of Q, enabling complex but numerically correct expressions (F7).

The key insight is that the target formula $\exp_q(0.5 \cdot dt \cdot \omega)$ needs only: $(*)$ for scalar product, $scale_v$ for scalar-vector product, and $\exp_q$ for the quaternion exponential. All other operations (add_v , q_mul , $norm_v$) provide extra expressive power that the search exploits before finding the simpler correct form. Curating the function pool based on the expected structure of the solution is a crucial design choice.

6.3 Constant Optimizer Behaviour

Even with the correct structural form, the search frequently finds trees with multiple sign-negation layers (e.g., $neg(neg(-0.5))$, $neg_v(neg_v(\omega))$). These arise because the Metropolis search freely mutates sign operators and the gradient-based constant optimizer adjusts values to compensate. Functionally, these trees are identical to $\exp_q(0.5 \cdot dt \cdot \omega)$, but they are visually noisy. The `simplifyTree` post-processing pass (iteration 5) absorbs $neg(c)$ into the constant value, reducing one layer of negation. Full normalization (eliminating $neg_v(neg_v(\cdot))$ chains) requires structural GADT rewrites with existential types, which are more complex to implement in Haskell and are left for future work.

6.4 Speed of Convergence

All successful runs completed in well under 300 seconds (usually within a few seconds to tens of seconds). The combination of 12 parallel-tempering chains and a 300-point subsample is highly effective. The high-temperature chains escape local minima, and the low-temperature chains refine constants once the correct structural form is found. This suggests that the typed approach eliminates enough of the search space that the remaining MCMC exploration is tractable even for a non-convex landscape.

6.5 Failed Experiments and Negative Results

- Depth=3 alone (iteration 2) was insufficient to eliminate F6; the root cause was the `add_v` operator, not tree depth.
- The `simplifyTree` pass did not fully normalise trees with multiple `neg_v` layers due to GADT type constraints; this is a limitation of the Haskell implementation rather than the method.
- The `scTargetEnergy=5e-5` threshold was reached by F7 trees (complex `q_mul` chains), showing that energy alone is not sufficient to distinguish correct from complex-but-equivalent trees. Structure inspection is also necessary.

7. Conclusion

We presented a typed symbolic regression system built in Haskell using a GADT expression tree, applied to discovering the quaternion integration formula $\Delta q = \exp_q(0.5 \cdot dt \cdot \omega)$ from synthetic motion capture data. The key contributions are:

1. **Multi-typed GADT trees:** All nodes are statically typed, eliminating ill-typed expressions from the search space by construction. No post-hoc type checking is needed.
2. **Minimal function pool design:** We showed that reducing the pool to only the operators needed for the target formula (from $\{(), scale_v, add_v, q_mul, exp_q, neg_v, neg\}$ to $\{(), scale_v, exp_q, neg_v, neg\}$) eliminates the three observed failure modes (F6, F7) without preventing the correct formula from being found.
3. **Parallel tempering on tree space:** 12 chains at log-spaced temperatures from 0.001 to 10.0, with 200 Metropolis steps between swaps, reliably finds the target formula. With the final function pool, the success rate is 10/10 (100%) across independent runs.

4. **Iterative failure mode analysis:** We identified two new failure modes (F6: extra dt factor; F7: complex quaternion product) by running the search, inspecting found trees algebraically, and tracing how each operator enabled degenerate solutions. This analysis directly guided pool design.

The iterative research loop — run, analyse, fix one thing, repeat — was essential. Early runs succeeded 67% of the time; by iteration 4, success was 100%. Each iteration provided a concrete algebraic diagnosis of what went wrong, making the fixes targeted rather than heuristic.

Future work includes: testing on real CMU mocap data with sensor noise, extending the type system to Lie algebra elements and rotation matrices, implementing full algebraic normalisation via e-graph equality saturation adapted for multi-sorted terms, and applying the approach to other physics-structured discovery problems (e.g., identifying Hamiltonian structure from trajectory data).